

Executive Recommendation: Re-scope the DevOps Transformation

Decision requested

Re-scope the program. Do not stop all migration work, and do not speed up every workstream just to meet a date.

Continue moving low-risk repositories where the new path is understood. Move more people and attention to pipeline conversion, legacy build support, security readiness, and reducing developer friction.

The reason is straightforward: repository migration is ahead, but pipelines are behind. A repository in GitHub is not useful if its build and test process is not reliable, its release artifacts are missing, or developers cannot get help. At the same time, disputed analytics should not be treated as a performance score.

What leaders should prioritize in the next 90 days

1. Restore confidence in pipeline conversion

Create an inventory of repository and pipeline owners, business importance, build tools, dependencies, security needs, artifacts, and runner requirements. Group pipelines into common patterns so teams can use reusable workflows rather than convert every pipeline from scratch. Pilot approximately 20–30 representative repositories, including standard workloads and difficult Windows/x86 cases.

For important services, run the old and new validation paths side by side for a limited period. Compare builds, tests, coverage, and output artifacts. Keep only one path authorized to deploy to production at a time. Retire the old path only after the service owner approves the new one and rollback has been practiced.

2. Make security stronger without making teams work around it

Turn on essential protections first: company identity, least-privilege workflow permissions, protected branches, code review, secret and dependency scanning, approved workflow actions, production approvals, and appropriate audit records. Before making a check mandatory across the organization, confirm that it is reliable, understandable, and supported.

Provide a quick exception route for products that cannot yet use the standard runner or workflow. Each exception must have an owner, reason, compensating protections, approver, expiry date, and planned fix. Review exceptions regularly. An exception is a managed temporary risk—not an undocumented bypass.

3. Reduce developer friction

Ask teams where the new process slows them down. Measure time to receive build feedback, repeated failure causes, blocked pull requests, and developer satisfaction. Improve the most common problems in the shared templates, documentation, and support process. Provide office hours and a clear place to ask for help. Treat frequent bypasses or failed checks as evidence that the platform needs attention.

4. Establish trustworthy DORA reporting before setting targets

Agree on what counts as a production deployment, which commits belong to it, which incidents count as deployment failures, and which timestamps define restoration. Show metric coverage, freshness, and known data problems next to reported values. Do not rank teams or individuals using data that is incomplete or not comparable.

The supplied workbook contains 164 commits, 350 build records, 11 incidents, and two projects. The link between commit `build_sk` and build `build_id` must be confirmed. Build records do not clearly show successful production deployments or build outcomes. Three incidents are marked deployment-caused, and one incident has a resolution timestamp earlier than its start. This is useful profiling evidence, but it is not a defensible full DORA baseline without source-system clarification and production deployment events.

Investment choices

Fund a small platform product team to build shared workflows, migration automation, documentation, and developer support. Fund secure Windows/x86 runner capacity for approved legacy products. Assign security and identity specialists to the rollout, and data engineering and stewardship capacity to Fabric and metric quality. Allow limited dual-running costs where they reduce release risk, with a clear end date and exit criteria.

Defer mass rewrites, forced Azure Boards migration, a large self-hosted runner fleet for every workload, universal mandatory checks before they are supported, team-level DORA targets, and AI agents that merge or deploy without human approval.

Decisions needed from executives

- Name the executive sponsor, platform product owner, service owners, and analytics steward.
- Approve service-based migration waves and the rule that pipeline parity and rollback must be demonstrated before old pipelines are retired.
- Approve the minimum security baseline, staged enforcement, exception approvers, and funding for isolated legacy runners.
- Confirm that Azure Boards remains in one ADO organization and name the owner for GitHub-to-Boards integration.
- Approve access to authorized analytics data, metric ownership, and the policy that DORA results will not be used to rank teams until data quality is acceptable.
- Agree on go/no-go measures: build and test reliability, artifact parity, security, rollback readiness, developer friction, runner cost, and metric quality.

What success looks like at day 90

A representative set of standard and legacy pipelines has passed agreed checks. Teams know who owns each repository and pipeline. The highest-risk runner exceptions have a supported route and expiry. Developers can report and see improvements to common friction. Leaders can review an initial analytics view that clearly shows definitions, coverage, and data limitations. The next migration wave is approved based on evidence—not repository counts alone.

Important boundary

This recommendation is a proposed plan based on the case-study scenario. The sample code, GitHub workflow, and Fabric notebook require review and execution in the candidate's environment. No enterprise migration, hosted workflow run, or Fabric workspace deployment is claimed here.